Gemini

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Image paths - 'V KOHLI.jpeg' is available in your Colab environment.
# You might need to upload 'image2.jpg' if you want a different image for the second operand.
img1_path = "V KOHLI.jpeg"
img2_path = "image2.jpg" # Placeholder for a second image to be uploaded

try:
    img1 = cv2.imread(img1_path)
    if img1 is None:
        raise FileNotFoundError(f"Image not found at {img1_path}. Please ensure it's uploaded to your

    img2 = cv2.imread(img2_path)
    if img2 is None:
        print(f"Warning: '{img2_path}' not found. Creating a placeholder image for img2.")
        # Create a simple colored rectangle as a placeholder for img2
        # Its dimensions should be smaller than img1 for the ROI operation to make sense
        img2 = np.zeros((100, 150, 3), dtype=np.uint8)
        img2[:, :, 0] = 255 # Blue channel
        img2[:, :, 1] = 0   # Green channel
        img2[:, :, 2] = 0   # Red channel (makes it blue)
        cv2.putText(img2, "IMG2", (20, 60), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
        # Resize img1 to be big enough to contain img2 if it's too small initially
        if img1.shape[0] < img2.shape[0] + 50 or img1.shape[1] < img2.shape[1] + 50:
            print("Note: img1 is too small for img2 placeholder. Resizing img1 to fit.")
            img1 = cv2.resize(img1, (max(img1.shape[1], img2.shape[1] + 100), max(img1.shape[0], img2


    rows, cols, channels = img2.shape
    # Ensure ROI coordinates are within img1 bounds
    start_row, start_col = 50, 50
    end_row, end_col = start_row + rows, start_col + cols
```

```
    if end_row > img1.shape[0] or end_col > img1.shape[1]:
        print("Warning: img2 is too large for the specified ROI in img1. Adjusting ROI or resizing im
        # Resize img2 to fit within img1's dimensions starting from (50,50)
        max_rows = img1.shape[0] - start_row
        max_cols = img1.shape[1] - start_col
        if max_rows <= 0 or max_cols <= 0: # If img1 is too small even for an ROI
            raise ValueError("Main image (img1) is too small for the specified ROI and img2 dimension

        scale_factor = min(max_rows / rows, max_cols / cols)
        img2 = cv2.resize(img2, (int(cols * scale_factor), int(rows * scale_factor)))
        rows, cols, channels = img2.shape
        end_row, end_col = start_row + rows, start_col + cols
        print(f"img2 resized to {rows}x{cols} to fit.")

    roi = img1[start_row:end_row, start_col:end_col]

    img2gray = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
    ret, mask = cv2.threshold(img2gray, 10, 255, cv2.THRESH_BINARY)
    mask_inv = cv2.bitwise_not(mask)

    img1_bg = cv2.bitwise_and(roi, roi, mask=mask_inv)
    img2_fg = cv2.bitwise_and(img2, img2, mask=mask)
    dst = cv2.add(img1_bg, img2_fg)

    img1[start_row:end_row, start_col:end_col] = dst

    # Convert BGR to RGB for displaying with matplotlib
    img1_rgb = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)

    # Display the result
    plt.figure(figsize=(10, 8))
    plt.imshow(img1_rgb)
    plt.title('Resulting Image (Image Overlay)')
    plt.axis('off')
    plt.show()

    cv2.imwrite('result_image.jpg', img1)
```

```
    print("Result image saved as 'result_image.jpg'")

except FileNotFoundError as e:
    print(f"Error: {e}")
except ValueError as e:
    print(f"Configuration Error: {e}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

```
Warning: 'image2.jpg' not found. Creating a placeholder image for img2.
```

Resulting Image (Image Overlay)